

Report on Complex Expressions Evaluation in C

Objective

The task was to calculate complex mathematical expressions using standard C mathematical functions. The program was implemented in C, and debugging was performed using the GNU Debugger (GDB) to verify the correctness of the calculations.

Code

Set 1

```
#include <stdio.h>
#define _USE_MATH_DEFINES
#include <math.h>

int main()
{
    // Set 1:
    double a = 2;
    double b = M_PI / 9;
    double c_var = 1;
    double d = 1;

    // expr_1: -0.012724451117138685
    // expr_2: 1.5533095657752118
    // result: -0.008191832071019456

    double expr_1 = (exp(tan(a))) + log(sin(b) + cos(c_var));
    printf("expr_1: %f\n", expr_1);
    double expr_2 = pow(pow(a, 3) + pow(b, 3) + pow(c_var, 3), 1.0 / 5.0);
    printf("expr_2: %f\n", expr_2);
    double result = expr_1 / expr_2;
    printf("Result: %f\n", result);

    // double expr_1 = (exp(tan(a))) + log(sin(b) + cos(c_var));
    // double expr_2 = pow(pow(a, 3) + pow(b, 3) + pow(c_var, 3), 1.0 / 5.0);
    // double result = expr_1 / expr_2;
    // printf("Result: %f\n", result);
    return 0;
}
```

Debugging with GDB

```
s@Notebook:~/programmin-part1/lab5$ gcc -g -O0 main.c -o main -lm
s@Notebook:~/programmin-part1/lab5$ gdb ./main
```

```

GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

```

```

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./main...
(gdb) b main
Breakpoint 1 at 0x1215: file main.c, line 8.
(gdb) r
Starting program: /home/s/programmin-part1/lab5/main

```

```

This GDB supports auto-downloading debuginfo from the following URLs:
    <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

```

```

Breakpoint 1, main () at main.c:8
8         double a = 2;
(gdb) n
9         double b = M_PI / 9;
(gdb) n
10        double c_var = 1;
(gdb) n
11        double d = 1;
(gdb) n
13        double expr_1 = (exp(tan(a))) + log(sin(b) + cos(c_var));
(gdb) n
14        printf("expr_1: %f\n", expr_1);
(gdb) n
expr_1: -0.012724
15        double expr_2 = pow(pow(a, 3) + pow(b, 3) + pow(c_var, 3), 1.0 / 5.0);
(gdb) n

```

```

16          printf("expr_2: %f\n", expr_2);
(gdb) n
expr_2: 1.553310
17          double result = expr_1 / expr_2;
(gdb) n
18          printf("Result: %f\n", result);
(gdb) p expr_1
$1 = -0.012724451117138685
(gdb) p expr_2
$2 = 1.5533095657752118
(gdb) p result
$3 = -0.0081918320710194555
(gdb) exit
A debugging session is active.

```

Inferior 1 [process 8828] will be killed.

Quit anyway? (y or n) y

Set 2

```

#include <stdio.h>
#define _USE_MATH_DEFINES
#include <math.h>

int main()
{
    // Set 2:
    double a = 1;
    double b = M_PI / 6;
    double c_var = 2;
    double d = 3;

    // expr_1: -0.012724451117138685
    // expr_2: 1.5533095657752118
    // result: -0.008191832071019456

    double expr_1 = (exp(tan(a))) + log(sin(b) + cos(c_var));
    printf("expr_1: %f\n", expr_1);
    double expr_2 = pow(pow(a, 3) + pow(b, 3) + pow(c_var, 3), 1.0 / 5.0);
    printf("expr_2: %f\n", expr_2);
    double result = expr_1 / expr_2;
    printf("Result: %f\n", result);

    // double expr_1 = (exp(tan(a))) + log(sin(b) + cos(c_var));
    // double expr_2 = pow(pow(a, 3) + pow(b, 3) + pow(c_var, 3), 1.0 / 5.0);

```

```

    // double result = expr_1 / expr_2;
    // printf("Result: %f\n", result);
    return 0;
}

```

Debugging with GDB

```

s@Notebook:~/programmin-part1/lab5$ gcc -g -O0 main2.c -o main -lm
s@Notebook:~/programmin-part1/lab5$ gdb ./main
Command 'gbd' not found, did you mean:
  command 'gnd' from snap gnd (master)
  command 'rbd' from snap microceph (18.2.4+snapc9f2b08f92)
  command 'gdb' from deb gdb (14.1-0ubuntu1)
  command 'gld' from deb postfix-gld (1.7-9)
  command 'gid' from deb id-utils (4.6.28-20200521ss15dab)
  command 'bd' from deb bd (1.03-3)
  command 'zbd' from deb zbd-utils (2.0.4-1)
  command 'gsd' from deb python3-gsd (3.0.1-3build1)
  command 'god' from deb ruby-god (0.13.7-3build5)
  command 'dbd' from deb netatalk (3.1.15~ds-1ubuntu1)
  command 'gbp' from deb git-buildpackage (0.9.33)
  command 'sbd' from deb sbd (1.5.1-2ubuntu1)
  command 'gzbd' from deb zbd-utils (2.0.4-1)
  command 'rbd' from deb ceph-common (19.2.0-0ubuntu0.24.04.1)
See 'snap info <snapname>' for additional versions.
s@Notebook:~/programmin-part1/lab5$ gdb ./main
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./main...
(gdb) b main
Breakpoint 1 at 0x1215: file main2.c, line 8.
(gdb) r
Starting program: /home/s/programmin-part1/lab5/main

```

This GDB supports auto-downloading debuginfo from the following URLs:
 <<https://debuginfod.ubuntu.com>>
 Enable debuginfod for this session? (y or [n]) y
 Debuginfod has been enabled.
 To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
 Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
 [Thread debugging using libthread_db enabled]
 Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

```
Breakpoint 1, main () at main2.c:8
8      double a = 1;
(gdb) n
9      double b = M_PI / 6;
(gdb) n
10     double c_var = 2;
(gdb) n
11     double d = 3;
(gdb) n
17     double expr_1 = (exp(tan(a))) + log(sin(b) + cos(c_var));
(gdb) n
18     printf("expr_1: %f\n", expr_1);
(gdb) n
expr_1: 2.267813
19     double expr_2 = pow(pow(a, 3) + pow(b, 3) + pow(c_var, 3), 1.0 / 5.0);
(gdb) n
20     printf("expr_2: %f\n", expr_2);
(gdb) n
expr_2: 1.556765
21     double result = expr_1 / expr_2;
(gdb) n
22     printf("Result: %f\n", result);
(gdb) n
Result: 1.456748
28     return 0;
(gdb) p expr_1
$1 = 2.2678129818252062
(gdb) p expr_2
$2 = 1.5567645942446764
(gdb) p result
$3 = 1.4567475328057047
(gdb) exit
A debugging session is active.
```

Inferior 1 [process 9857] will be killed.

```
Quit anyway? (y or n) y
s@Notebook:~/programmin-part1/lab5$
```

Set 3

```
#include <stdio.h>
#define _USE_MATH_DEFINES
#include <math.h>

int main()
{
#include <stdio.h>
#define _USE_MATH_DEFINES
#include <math.h>

int main()
{
    // Set 3:
    double a = 3;
    double b = M_PI / 3;
    double c_var = 4;
    double d = 5;

    // expr_1: -0.6822
    // expr_2: 2.4711
    // result: -0.2761

    double expr_1 = (exp(tan(a))) + log(sin(b) + cos(c_var));
    printf("expr_1: %f\n", expr_1);
    double expr_2 = pow(pow(a, 3) + pow(b, 3) + pow(c_var, 3), 1.0 / 5.0);
    printf("expr_2: %f\n", expr_2);
    double result = expr_1 / expr_2;
    printf("Result: %f\n", result);

    // double expr_1 = (exp(tan(a))) + log(sin(b) + cos(c_var));
    // double expr_2 = pow(pow(a, 3) + pow(b, 3) + pow(c_var, 3), 1.0 / 5.0);
    // double result = expr_1 / expr_2;
    // printf("Result: %f\n", result);
    return 0;
}

}
```

Debugging with GDB

```
s@Notebook:~/programmin-part1/lab5$ gcc -g -O0 main3.c -o main -lm
```

```
s@Notebook:~/programmin-part1/lab5$ gdb ./main
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.
```

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./main...
(gdb) b main
Breakpoint 1 at 0x1215: file main3.c, line 8.
(gdb) r
Starting program: /home/s/programmin-part1/lab5/main
```

```
This GDB supports auto-downloading debuginfo from the following URLs:
    <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
```

```
Breakpoint 1, main () at main3.c:8
8         double a = 3;
(gdb) n
9         double b = M_PI / 3;
(gdb) n
10        double c_var = 4;
(gdb) n
11        double d = 5;
(gdb) n
17        double expr_1 = (exp(tan(a))) + log(sin(b) + cos(c_var));
(gdb) n
18        printf("expr_1: %f\n", expr_1);
(gdb) n
expr_1: -0.682223
19        double expr_2 = pow(pow(a, 3) + pow(b, 3) + pow(c_var, 3), 1.0 / 5.0);
(gdb) n
```

```

20         printf("expr_2: %f\n", expr_2);
(gdb) n
expr_2: 2.471141
21         double result = expr_1 / expr_2;
(gdb) n
22         printf("Result: %f\n", result);
(gdb) n
Result: -0.276076
28         return 0;
(gdb) p expr_1
$1 = -0.6822225674285749
(gdb) p expr_2
$2 = 2.4711410860694731
(gdb) p result
$3 = -0.27607592754394233
(gdb) exit
A debugging session is active.

```

Inferior 1 [process 10487] will be killed.

```

Quit anyway? (y or n) y
s@Notebook:~/programmin-part1/lab5$

```

Debugging with GDB

The debugging process was conducted using the GNU Debugger (GDB). Key steps included:

1. Setting Breakpoints: A breakpoint was set at the beginning of the main function.
2. Stepping Through the Code: The program was executed line by line using the n (next) command to inspect intermediate values.
3. Inspecting Variables: The p command was used to print the values of expr_1, expr_2, and result at runtime.
4. Verification: The outputs were compared to the expected results for accuracy.

Conclusion

The program successfully calculates the complex expressions for multiple sets of inputs. Debugging with GDB confirmed the correctness of the results and provided a deeper understanding of the program's execution. This project demonstrates proficiency in:

1. Using standard C mathematical functions.
2. Writing efficient and readable code.
3. Debugging with GDB to validate and troubleshoot the program.